

Answers:

General

1. Order these from most compact to least compact to represent, and provide reasoning: Quaternions, Euler Angles, Rotation Matrices
 - a. Euler angles - 3 values, Quaternions - 4 values, Rotation Matrices - 9 values
2. How are quaternions generally represented? Recall that complex numbers are represented as $a + bi$
 - a. $a + bi + cj + dk$ where a, b, c, d are real numbers
3. What advantages do quaternions hold over rotation matrices & euler angles?
 - a. They're more compact than matrices, don't suffer from Gimbal lock like Euler angles, and can be smoothly interpolated.
4. When games store quaternions in a 4D vector, where is the "real" part typically stored?
 - a. In the w component. So if a quaternion is represented as

$$a + bi + cj + dk$$

Where a is the coefficient for the real portion, and b, c, d are the coefficients for the imaginary portion, then most games will store this in a 4D vector like this (b, c, d, a)

5. Why might you still choose to use Euler angles over quaternions?
 - a. In a lot of situations, it's beneficial to use a simpler format like Euler angles for a variety of reasons. They're quite readable and easy to understand, and they're also easy to work with. In user interfaces, for example, having a more intuitive format would serve users such as designers better. In code, in situations where you know ahead of time that the rotations will be constrained to 1 or 2 axes, such as say, a character that's walking, they may only rotate around a single axis, in which case, an Euler angle representation is sufficient.
6. What are a , and i, j, k in a quaternion?
 - a. a is the real unit while i, j, k are the imaginary portion
7. What advantages do matrices hold over quaternions?
 - a. Easy to understand at a glance and visualize intuitively in terms of basis vectors. They're also more versatile, able to represent not just rotations but other linear transformations. Finally, matrices can be composed easily through simple multiplication, and although quaternions can also be composed via multiplication, the process is more complex.

Axis Angle

8. What is the basic formula for constructing a quaternion, given an angle θ and a rotation axis $\hat{n}$.
- a. $q = \cos(\theta/2) + \sin(\theta/2)(\hat{n}_x i + \hat{n}_y j + \hat{n}_z k)$
9. True/False: Quaternions can be directly calculated from an axis-angle representation
- a. True
10. True/False: You need to normalize the vector component of a quaternion in order to normalize a quaternion.
- a. False, the real component must also be used.
11. True/False: A quaternion and a 4D cartesian vector are normalized in the same way.
- a. True
12. True/False: In axis-angle, the axis is a unit vector
- a. True
13. True/False: The axis-angle representation makes smooth interpolation easy, vs difficulties interpolating Euler angles and rotation matrices.
- a. False, they're also difficult to interpolate.
14. True/False: A unit quaternion has length 1.
- a. True
15. Normalize the following quaternion: $q = 2 + 2i + 3j + 1k$
- a. First we need to $\|q\|$, so calculate the length:

$$\|q\| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

$$\|q\| = \sqrt{2^2 + 2^2 + 3^2 + 1^2}$$

$$\|q\| = \sqrt{4 + 4 + 9 + 1}$$

$$\|q\| = \sqrt{18} = 4.24...$$

$$q_{\text{normalized}} = q / \|q\| = 0.47 + 0.47i + 0.94j + 0.23k$$

16. Normalize the following quaternion: $q = 0.5 + 0.866i + 0j + 0k$

a. $\|q\| = \sqrt{a^2 + b^2 + c^2 + d^2}$

$$\|q\| = \sqrt{0.5^2 + 0.866^2}$$

$$\|q\| = \sqrt{0.25 + 0.75}$$

$$\|q\| = 1$$

$$q_{\text{normalized}} = q, \text{ it was already normalized}$$

17. Normalize the following quaternion: $q = 0.5 + 0.25i + 0.25j + 0k$

a. $\|q\| = \sqrt{a^2 + b^2 + c^2 + d^2}$

$$\|q\| = \sqrt{0.5^2 + 0.25^2 + 0.25^2}$$

$$\|q\| = \sqrt{0.25 + 0.0625 + 0.0625}$$

$$\|q\| = \sqrt{0.375} = 0.612...$$

$$q_{\text{normalized}} = q / \|q\| = 0.816 + 0.408i + 0.408j + 0k$$

18. If you have an axis $\hat{v} = (1, 0, 0)$ and an angle $\theta = 30^\circ$, construct a unit quaternion from this.

- a. So the formula for constructing a quaternion from axis-angle is

$$q = \cos(\theta/2) + \sin(\theta/2)(\hat{n}_x i + \hat{n}_y j + \hat{n}_z k)$$

so

$$\cos(30^\circ/2) = 0.97$$

$$\sin(30^\circ/2) = 0.26$$

Thus, $q = \mathbf{0.97 + 0.26i + 0j + 0k}$

19. If you have an axis $\hat{v} = (0.5, 0, 0.866)$ and an angle $\theta = \frac{\pi}{3}$ radians, construct a unit quaternion from this.

a. $\cos(\frac{\pi}{6} \text{ radians}) = 0.866$

$\sin(\frac{\pi}{6} \text{ radians}) = 0.5$

Thus, $q = \mathbf{0.866 + 0.25i + 0j + 0.433k}$

20. Why is linearly interpolating a vector difficult?

a. The interpolated vectors may get shortened, leading to an undesirable transition.

21. As a follow-up, if you normalize the vector after interpolating, is this sufficient to interpolate smoothly? If not, explain why.

a. You saw from examples that if the vectors face away from each other, for example, the interpolated vectors get shortened extensively, and if the 2 vectors were to face away from each other completely, you could end up with a 0 vector when interpolating between them. Additionally, we could see that as the vectors were interpolated, their spacing wasn't uniform. So although normalizing the vector corrects the vector shortening, it still doesn't preserve angular velocity.

22. Can you think of any cases where linear interpolation of vectors is acceptable?

a. Generally, you can get away with it still in a lot of cases where the expected difference in the vectors isn't that big. An easy example would be the normals of a triangle, these are typically interpolated and then renormalized in the fragment shader.

23. What is the relationship between the angle θ in an axis-angle representation, and it's corresponding quaternion?

a. In the quaternion built from the axis-angle representation, θ is halved and then the cosine and sin of $\theta/2$ is used to construct the quaternion.

Multiplication & Inverse

24. True/False: Multiplying quaternions allows you to compose several rotations together.

a. True, this is similar to how matrix multiplication allows you to compose linear transformations

25. True/False: The inverse of a unit quaternion is obtained by negating its components.
 a. False, for a unit quaternion, you obtain the inverse by negating the vector components.
26. True/False: Multiplying a quaternion by its inverse results in an identity quaternion.
 a. True
27. True/False: The identity quaternion is $\mathbf{0} + \mathbf{0i} + \mathbf{0j} + \mathbf{0k}$
 a. False: The identity is $\mathbf{1} + \mathbf{0i} + \mathbf{0j} + \mathbf{0k}$
28. True/False: Quaternion multiplication is commutative.
 a. False
29. Given $q_1 = \mathbf{1} + \mathbf{1i} + \mathbf{1j} + \mathbf{1k}$ and $q_2 = \mathbf{1} + \mathbf{0i} + \mathbf{0j} + \mathbf{0k}$, calculate q_1q_2
 a. For a quaternion $q = a + v_xi + v_yj + v_zk$

$$q_1q_2 = a_1a_2 - \text{dot}(v_1, v_2) + a_1v_2 + a_2v_1 + v_1 \times v_2$$

So

$$q_1q_2 = 1 - (1, 1, 1) \cdot (0, 0, 0) + (1, 1, 1)$$

$$q_1q_2 = 1 + 1i + 1j + 1k$$

30. Given $q_1 = \mathbf{1} + \mathbf{1i} + \mathbf{1j} + \mathbf{1k}$ and $q_2 = \mathbf{0} + \mathbf{0i} + \mathbf{1j} + \mathbf{0k}$, calculate q_1q_2
 a. For a quaternion $q = a + v_xi + v_yj + v_zk$

$$q_1q_2 = a_1a_2 - \text{dot}(v_1, v_2) + a_1v_2 + a_2v_1 + v_1 \times v_2$$

So

$$a_1 = 1$$

$$v_1 = (1, 1, 1)$$

$$a_2 = 0$$

$$v_2 = (0, 1, 0)$$

$$v_1 \times v_2 = (-1, 0, 1)$$

$$q_1q_2 = 0 - (1, 1, 1) \cdot (0, 1, 0) + 1(0, 1, 0) + 0(1, 1, 1) + (-1, 0, 1)$$

$$q_1q_2 = 0 - 1 + (0, 1, 0) + (0, 0, 0) + (-1, 0, 1)$$

$$q_1q_2 = -1 + (-1, 1, 1)$$

$$q_1q_2 = -1 - 1i + 1j + 1k$$

31. Given a quaternion $\hat{q} = \mathbf{a} + \mathbf{bi} + \mathbf{cj} + \mathbf{dk}$, calculate the inverse

- a. $\mathbf{a} - \mathbf{bi} - \mathbf{cj} - \mathbf{dk}$ since $\hat{q}$ is a unit quaternion.

32. Explain why negating the vector component of a unit quaternion is sufficient to invert it.

- a. Given a quaternion $\hat{q} = \mathbf{a} + \mathbf{bi} + \mathbf{cj} + \mathbf{dk}$, the $\mathbf{i}, \mathbf{j}, \mathbf{k}$ components are related to the axis of rotation, scaled by the sin of the angle. Negating this portion basically reverses the direction of rotation. Note, that this is only applicable to unit quaternions.

Transforming Vectors

33. True/False: Multiplying a quaternion by its inverse results in the zero quaternion
- False, it results in the identity quaternion
34. True/False: The process transforming a vector is referred to as the sandwich product.
- True
35. True/False: Multiplying a quaternion q by the identity quaternion only results in q if q is a unit quaternion.
- False, doesn't matter what q is.
36. True/False: You must "promote" a vector to a quaternion to transform it by a quaternion.
- True
37. True/False: Promoting a vector to a quaternion involves copying the xyz components of the vector into the ijk components of the quaternion.
- True
38. Explain the steps of transforming a vector v by a quaternion $\hat{q}$, in your own words.
- The steps are:
 - Promote the vector v to a quaternion by copying the xyz components into the ijk components of the quaternion, and setting 0 for the real portion. We can call this new quaternion q_v .

ie. $(x, y, z) \rightarrow (x, y, z, 0)$

 - Multiply the $\hat{q}v$, call the result q_{temp}
 - Multiply q_{temp} by the inverse of $\hat{q}$, so $q_{\text{final}} = q_{\text{temp}}\hat{q}^{-1}$
 - Copy the ijk component coefficients from q_{final} back into a vector.

ie. $(x, y, z, w) \rightarrow (x, y, z)$
39. Why do we use $\theta/2$ when defining a rotation quaternion?
- Because we use the sandwich product to effectively apply the rotation twice, meaning that the quaternion only needs to represent half the rotation.
40. What happens to rotations on the scalar real value when using the sandwich product?
- Rotations in those planes get cancelled out by the sandwich product part of the vector transformation, leaving only rotations in 3D space.
41. Explain why, when doing the sandwich product qvq^{-1} , the inverse quaternion q^{-1} doesn't cancel out the first rotation q .
- First thing to remember is that quaternion multiplication isn't commutative, $ab \neq ba$. So with that in mind, $qv \neq vq$, having q be on the left vs the right matters a lot. There are differences in how the rotation is applied. If you were to calculate qv , and compare it to vq , you'd see that the rotation in the $wx/wy/wz$ planes was the same, but reversed for the $xy/yz/zx$ planes. Remember, if you consider quaternions as 4D, then you have the $xy/yz/zx$ planes, but you have that w "real" component which forms 3 new planes $wx/wy/wz$, for a total of 6

planes.

By using the inverse on the right side, we're able to cancel out the rotational effects in the wx/wy/wz planes and isolate the rotation to only apply the rotation in 3D space.

42. Given a quaternion $\hat{q} = 0 + i + 0j + 0k$, what rotation is this performing when transforming a vector?
- This is a 180° rotation around the x-axis.

SLERP

43. True/False: Lerp can also be applied to quaternions.
- True, there's nothing stopping you from lerp'ing quaternions.
44. True/False: Renormalizing a vector after a lerp results in constant angular velocity.
- False
45. True/False: Slerp is more computationally expensive than lerp.
- True, lerp is extremely compact. In a SIMD type of setting, lerp can be implemented in a few instructions.
46. True/False: Lerp can sometimes be useful to use, even if Slerp is available.
- True, in any situation where a "good enough" answer is sufficient, such as say, you know that the angular differences between the quaternions is small. Performance also plays a role, since lerp is much more performant than slerp, it may be preferable to use in situations where the quality of the rotation isn't your primary concern.
47. True/False: The formula for lerp, given 2 values A, B, and a value t is $A + t * (B - A)$
- True
48. Explain what linear interpolation, or lerp, does in the context of a bar graph.
- In the context of a bar graph, lerp would find the value of a given point between 2 known values, A and B.
49. What problem do you suffer from when lerp'ing between 2 unit vectors?
- Lerp'ing between 2 unit vectors doesn't maintain the unit length property for the interpolated vectors. Additionally, the angular velocity of the interpolation isn't uniform. If you were to normalize the vectors, you'd see that you have uneven spacing between the ends of each vector.
50. Explain what angular velocity refers to, in the context of interpolating 2 unit vectors. Demonstrate with an example.
- So any answer that basically talks about how quickly the angle changes is going to be fine.

We can easily see linear interpolation not preserving angular velocity by simply plotting a few, evenly spaced interpolated vectors. Consider 2 vectors pointing nearly away from each other. If you were to plot the interpolated vectors for $t = 0$, $t = 0.1$, $t = 0.2$, etc... $t = 1$. Let's call those v_0 , $v_{0.1}$, $v_{0.2}$, etc...

And then you normalize each of those vectors to maintain unit length. Now if you take the first 2 vectors, v_0 and $v_{0.1}$, and look at the angle between them,

compare that to say, $v_{0.5}$, $v_{0.6}$. If the angular velocity was constant, we'd see exactly the same value. But we don't.

We can do some numbers.

$$a = (-0.98, 0.17)$$

$$b = (0.98, 0.17)$$

$$v_0 = (-0.98, 0.17)$$

$$v_{0.1} = \text{normalize}(-0.784, 0.17) = (-0.977, 0.212)$$

$$v_{0.5} = (0, 1)$$

$$v_{0.6} = \text{normaliz}(0.196, 0.17) = (0.755, 0.655)$$

$$\text{dot}(v_0, v_{0.1}) = 0.9935$$

$$\text{dot}(v_{0.5}, v_{0.6}) = 0.655$$

Clearly, the angle between them isn't similar